

Estudio Computacional de $A(n, d)$: Implementación, Cotas y Reproducibilidad

Amador Jesús, Hernández Roberto, Hernández Belinda, Ramos Sarai

May 25, 2026

Abstract

El número $A(n, d)$ es el máximo tamaño de un código binario de longitud n y distancia de Hamming mayor o igual a d este es un problema central en teoría de códigos. Su naturaleza $\mathcal{NP}$ -completa obliga a combinar algoritmos exactos, heurísticos y cotas teóricas. En este trabajo presentamos una implementación modular en Python que incluye: generación del grafo de Hamming, algoritmo de Tomita para cliques máximas, cotas de Hamming, Singleton, Plotkin y Gilbert–Varshamov, así como heurísticas como lo son búsqueda local y recocido simulado. Discutimos los desafíos de replicabilidad computacional y la importancia de alojar estos códigos en repositorios públicos, destacando nuestro progreso desde enfoques iniciales de backtracking hasta la adopción de métodos heurísticos para valores grandes de n .

1 Introducción

Un código binario $C \subseteq \mathbb{F}_2^n$ tiene distancia mínima d si para todo $x \neq y \in C$, $d_H(x, y) \geq d$. El parámetro $A(n, d)$ es la máxima cardinalidad de un tal código. Su determinación es fundamental para el diseño de sistemas de comunicación tolerantes a errores. Dado que el problema es $\mathcal{NP}$ -duro, no existe una fórmula cerrada. Para $n \leq 10$ se conocen valores exactos gracias a tablas clásicas [?, ?, ?], pero la replicación de esos resultados computacionalmente es un reto debido a la explosión combinatoria y a la falta de código abierto.

En nuestro proyecto hemos desarrollado una librería Python (`hammindist.functions`) que implementa:

- Generación del grafo de Hamming usando bitsets.
- Algoritmo exacto de Tomita para la clique máxima (con poda por coloración y reducción por paridad).
- Cotas superiores (Hamming, Singleton, Plotkin) y la cota inferior existencial de Gilbert–Varshamov.
- Heurísticas para $n > 10$: búsqueda local greedy con reinicios aleatorios y recocido simulado con memoria tabú.

Todo el código está disponible en un repositorio GitHub para garantizar la reproducibilidad.

2 Marco Teórico

2.1 Reducción a clique máxima

Definimos el grafo de Hamming $H(n, d) = (V, E)$ con $V = \mathbb{F}_2^n$ y $\{u, v\} \in E$ si $d_H(u, v) \geq d$. Un código válido es una clique en $H(n, d)$, por tanto

$$A(n, d) = \omega(H(n, d)),$$

donde $\omega(G)$ es el tamaño de la clique máxima. Esta reformulación permite usar algoritmos estándar de teoría de graficas.

2.2 Cotas superiores e inferiores

Las siguientes cotas son fundamentales en nuestro paquete.

- **Cota de Hamming (esferas):** Sea $t = \lfloor (d-1)/2 \rfloor$. Las bolas de radio t centradas en los codewords son disjuntas, luego

$$A(n, d) \leq \frac{2^n}{\sum_{i=0}^t \binom{n}{i}}.$$

Demostración: Si $x \in B(c, t) \cap B(c', t)$ con $c \neq c'$, entonces $d_H(c, c') \leq 2t < d$, contradicción. El volumen de cada bola es $\sum_{i=0}^t \binom{n}{i}$, y todas caben en $\mathbb{F}_2^n$, luego $M \cdot \sum_{i=0}^t \binom{n}{i} \leq 2^n$.

- **Cota de Singleton:** Proyectando las primeras $n-d+1$ coordenadas, dos palabras con igual proyección diferirían sólo en las últimas $d-1$ coordenadas, luego $d_H \leq d-1$, imposible. Por tanto la proyección es inyectiva y $A(n, d) \leq 2^{n-d+1}$.
- **Cota de Plotkin:** Si $2d > n$, entonces $A(n, d) \leq \lfloor \frac{2d}{2d-n} \rfloor$. *Demostración (esbozo):* Contando la suma S de distancias por pares: por un lado $S \geq \binom{M}{2}d$; por otro, cada coordenada contribuye $n_k(M-n_k) \leq M^2/4$, luego $S \leq nM^2/4$. Igualando se obtiene $(2d-n)M \leq 2d$, de donde la cota.
- **Cota de Gilbert–Varshamov (existencial):** Existe un código de tamaño al menos $\frac{2^n}{\sum_{i=0}^{d-1} \binom{n}{i}}$. *Demostración (constructiva):* Algoritmo greedy: añadir palabras mientras sea posible. Al terminar, cada palabra no usada está a distancia $\leq d-1$ de alguna ya elegida, luego las bolas de radio $d-1$ cubren $\mathbb{F}_2^n$; tomando cardinalidades se obtiene la cota.

Nuestra función `upper_bound` devuelve el mínimo entre las cotas de Hamming y Plotkin, y `sphere_packing_bound`, `plotkin_bound` implementan cada una respectivamente.

3 Implementación Computacional

3.1 Generación del grafo y bitsets

La función `build_adjacency_bitsets(n,d,even_only)` construye la matriz de adyacencia como una lista de enteros donde el bit j de `adj[i]` indica si el vértice i es adyacente a j . Usamos representación binaria directa: cada palabra es un entero de 0 a $2^n - 1$. La distancia de Hamming se calcula con `(x^y).bit_count()`. Para d par, la opción `even_only=True` reduce el número de vértices a la mitad (solo palabras de peso par), acelerando drásticamente la búsqueda.

3.2 Algoritmo de Tomita para clique máxima

La función `max_clique_tomita` implementa el algoritmo de Östergård[?] (versión de Tomita) con poda por coloración greedy. Mantiene tres conjuntos R (clique actual), P (candidatos) y X (prohibidos). En cada paso:

- Si $|R| + |P| \leq \text{max_size}$ se poda.
- Se colorea P de forma greedy (función `greedy_color`) y se obtiene una cota superior; si $|R| + \text{colores} \leq \text{max_size}$ se poda.
- Se elige un pivote en $P \cup X$ que maximice $|P \cap N(u)|$ y se itera sobre $P \setminus N(\text{pivote})$.

Tras la búsqueda, la función devuelve el tamaño y la clique encontrada.

3.3 Heurísticas para n grande

Para $n > 10$, el algoritmo exacto es inviable. Desarrollamos dos heurísticas:

- **Búsqueda local greedy con reinicios:** `greedy_start` baraja aleatoriamente las palabras y las añade si son compatibles; se repite un número de iteraciones (por defecto 1000). La mejor solución se guarda como cota inferior.
- **Recocido simulado con memoria tabú:** `simulated_annealing` realiza perturbaciones (elimina entre 1 y 3 palabras e intenta añadir una más), acepta soluciones peores con probabilidad $e^{\Delta/T}$, y mantiene una lista tabú para evitar ciclos. La temperatura se enfría geométricamente.

Estas heurísticas permiten obtener cotas inferiores competitivas para $n = 12, 13, \dots$ y son modulares para futuras mejoras.

4 Resultados y Discusión

Validamos nuestro algoritmo exacto con valores conocidos (Cuadro 1).

Para valores de $n \leq 8$ y $d \geq 4$ el algoritmo de búsqueda de clanes obtiene de forma razonable los resultados, sin embargo para valores como $A(8,3)=A(9,4)$ o $A(10,4)$ este resulta inviable

Table 1: Cotas superiores e inferiores para $A(n, d)$ (valores seleccionados)

n	$d = 4$	$d = 6$	$d = 8$	$d = 10$	$d = 12$
6	4	2	1	1	1
7	8	2	1	1	1
8	16	2	2	1	1
9	20	4	2	1	1
10	40	6	2	2	1
11	72	12	2	2	1
12	144	24	4	2	2
13	256	32	4	2	2
14	512	64	8	2	2
15	1024	128	16	4	2
16	2048	256	32	4	2
17	2816–3276	258–340	36	6	2
18	5632–6552	512–673	64	10	4
19	10496–13104	1024–1237	128	20	4
20	20480–26168	2048–2279	256	40	6

Fuentes: [1,2,3]. Para $n > 20$, consultar referencias originales.

pues tarda dias ejecutando el codigo sin resultados, por lo que, mediante el uso de valores conocidos de $A(n, d)$ obtamos por utilizar un enfoque heuristico para abordar el problema.

Un aspecto crítico que hemos observado es la **dificultad de replicar resultados computacionales** publicados en la literatura. Muchas tablas de $A(n, d)$ se obtuvieron con algoritmos no disponibles públicamente, con parámetros de poda o heurísticas no documentadas. Por ello, consideramos esencial alojar nuestro código en un repositorio GitHub (url: <https://github.com/JesusAmador25/hammindist>) junto con instrucciones claras para su instalación y ejecución. Esto no solo garantiza la transparencia de nuestros propios resultados, sino que permite a otros investigadores verificar, extender o aplicar nuestras funciones.

5 Conclusión y Trabajo Futuro

Hemos desarrollado una herramienta computacional completa para estimar $A(n, d)$ que combina algoritmos exactos (Tomita) con heurísticas y cotas teóricas. El paquete está documentado y es reproducible. Como trabajo futuro, planeamos:

- Incorporar la cota de Delsarte mediante programación lineal.
- Paralelizar el algoritmo de Tomita usando múltiples núcleos.
- Extender el estudio a códigos no binarios y a otras métricas.

Todo el código y el presente reporte se encuentran en el repositorio, fomentando la colaboración y la reproducibilidad en matemática computacional.

References

- [1] M. R. Best, A. E. Brouwer, F. J. MacWilliams, A. M. Odlyzko, N. J. A. Sloane. Bounds for binary codes of length less than 25. *IEEE Transactions on Information Theory*, 24(1):81–93, 1978.
- [2] E. Agrell, A. Vardy, K. Zeger. A table of upper bounds for binary codes. *IEEE Transactions on Information Theory*, 47(7):3004–3006, 2001.
- [3] A. E. Brouwer. Bounds for binary codes. Tablas en línea: <https://www.win.tue.nl/~aeb/codes/binary-1.html>, 2018.
- [4] F. J. MacWilliams, N. J. A. Sloane. *The Theory of Error-Correcting Codes*. North-Holland, Amsterdam, 1977.
- [5] P. Delsarte. An algebraic approach to the association schemes of coding theory. *Philips Research Reports Supplements*, No. 10, 1973.
- [6] A. Schrijver. New code upper bounds from the Terwilliger algebra and semidefinite programming. *IEEE Transactions on Information Theory*, 51(8):2859–2866, 2005.
- [7] C. Bron, J. Kerbosch. Algorithm 457: Finding All Cliques of an Undirected Graph. *Communications of the ACM*, 16(9):575–577, 1973.
- [8] E. Tomita, A. Tanaka, H. Takahashi. The worst-case time complexity for generating all maximal cliques. *Theoretical Computer Science*, 363(3):260–267, 2006.
- [9] P. R. J. Östergård. A fast algorithm for the maximum clique problem. *Discrete Applied Mathematics*, 120(1-3):197–207, 2002.
- [10] Sage Developers. *SageMath Documentation: Coding Theory*. <https://doc.sagemath.org/html/en/reference/coding/>, 2024.